Rapport de Travaux Pratiques

Securite des bases de donnees NoSQL

MongoDB – Authentification, RBAC & Audit

Etudiante : Asma Daoui

GitHub : BeeHope **Email :** asmadaoui6@gmail.com

Formation : Ingenierie Cybersecurite – 4eme annee

Etablissement : EDIA – Universite Euro-Mediterraneenne de Fes (UEMF)

Date : 5 avril 2026

Résumé

Ce rapport documente les travaux pratiques portant sur la securisation d'une instance MongoDB deployee via Docker. Trois activites ont ete realisees : (1) demonstration concrete de l'acces non authentifie versus authentifie, (2) configuration de l'authentification SCRAM et verification du refus d'acces anonyme, et (3) implementation d'une politique de controle d'acces base sur les roles (RBAC) avec creation d'un role personnalise et tests autorise/refuse pour chaque compte applicatif. Toutes les etapes sont documentees par des captures d'ecran commentees et analysees en detail.

Table des matières

1 Activite 1 – Instance insecure vs instance secure **2**

1.1 Objectif et contexte 2

1.2 Capture 1 – Lancement de l'instance insecure et echec du port 2

1.3 Capture 2 – Connexion anonyme et insertion sans authentification . . . 4

1.4 Capture 3 – Lancement de l'instance secure et connexion anonyme refusee 6

1.5 Capture 4 – Connexion authentifiee avec listDatabases reussie 7

1.6 Checkpoint 1 – Reponses 8

2 Activite 2 – Authentification et preuve de refus **10**

2.1 Objectif 10

2.2 Capture 1 – Refus anonyme et connexion admin avec getUsers 11

2.3 Capture 2 – Verification de l'authenticationDatabase 13

2.4 Checkpoint 2 – Reponses 14

3 Activite 3 – Politique RBAC **16**

3.1 Objectif et architecture RBAC 16

3.2 Capture 1 – Insertion des donnees de test et creation du role 17

3.3 Capture 2 – Insertion des incidents, users et creation des comptes . . . 19

3.4 Capture 3 – Creation du role et des trois comptes, db.getUsers() 21

3.5 Capture 4 – db.getUsers() complet avec les 3 comptes 23

3.6 Capture 5 – Tests soc_analyst_ro (autorise et refuse) 25

3.7 Capture 6 – Tests soc_manager (autorise et refuse) 27

3.8 Capture 7 – Tests soc_app (autorise et refuse) 29

3.9 Checkpoint 3 – Reponses 30

4 Synthese – Matrice des droits et recommandations **32**

4.1 Matrice des droits RBAC observes 32

4.2 Recommandations de securite 32

4.3 Auto-check final 33

1. Activite 1 – Instance insecure vs instance secure

1.1 Objectif et contexte

L'objectif de cette activite est de constater experimentalement la difference de comportement entre une instance MongoDB lancee **sans authentification** et une instance lancee **avec authentification**. Il s'agit de la premiere etape d'une demarche de securisation progressive : avant de corriger une vulnerabilite, il faut la rendre visible et mesurable.

MongoDB, par defaut dans certaines configurations historiques, n'active pas l'authentification. Cela signifie que n'importe quel client reseau capable d'atteindre le port 27017 peut lire, ecrire et administrer toutes les bases de donnees sans fournir le moindre identifiant. Ce scenario a ete massif en production entre 2016 et 2020, conduisant a des rancons et des fuites de donnees a grande echelle.

1.2 Capture 1 – Lancement de l'instance insecure et echec du port

```
PS C:\Users\HP> docker rm -f mongo_insecure
mongo_insecure
PS C:\Users\HP> docker run -d --name mongo_insecure -p 27018:27017 mongo:7
c86acea62c53d29e045ed976a5523414b557fb5e08452b9d2ffba12c89e26f19
docker: Error response from daemon: failed to set up container networking: driver failed programming external connectivity on endpoint mongo_insecure (6cae48
69ca37eafd9e2d6433df45dc37c657c7047f35dead3b455d746f1a6f0a): Bind for 0.0.0.0:27018 failed: port is already allocated
```

FIGURE 1.1 – Tentative de lancement de `mongo_insecure` avec conflit de port

Capture n° 1 – Preuve a conserver

La figure 1.1 montre la commande de lancement de l'instance insecure avec le flag `-p 27018:27017`. Docker retourne l'erreur :

```
Bind for 0.0.0.0:27018 failed: port is already allocated
```

Analyse – Conflit de port – cause et correction

Cette erreur survient parce qu'un autre processus ou conteneur occupe deja le port 27018 sur l'hote. Dans un environnement Docker sur Windows, cela peut etre du a une instance precedente non supprimee ou a un processus systeme. La solution est d'utiliser `docker rm -f` pour supprimer le conteneur precedent avant de relancer, ou de choisir un port hote different. Ce premier obstacle demontre l'importance du nettoyage preventif avant chaque deploiement de laboratoire.
D'un point de vue securite, le fait que Docker publie par defaut sur `0.0.0.0` (toutes les interfaces) est notable : cela expose directement le service MongoDB sur le reseau local, ce qui est problematique pour une instance sans authentification.

1.3 Capture 2 – Connexion anonyme et insertion sans authentification

```
test>

test> use soc_lite
switched to db soc_lite
soc_lite> db.security_logs.insertOne({
|   timestamp: new Date(),
|   source_ip: "10.0.0.99",
|   event_type: "test_insecure",
|   message: "Inserted without authentication"
| })
{
  acknowledged: true,
  insertedId: ObjectId('69c5064f55055c9a4fd805db')
}
soc_lite> db.security_logs.find({ event_type: "test_insecure" })
[
  {
    _id: ObjectId('69c5064f55055c9a4fd805db'),
    timestamp: ISODate('2026-03-26T10:11:27.127Z'),
    source_ip: '10.0.0.99',
    event_type: 'test_insecure',
    message: 'Inserted without authentication'
  }
]
soc_lite>

soc_lite> db.security_logs.find({ event_type: "test_insecure" })
[
  {
    _id: ObjectId('69c5064f55055c9a4fd805db'),
    timestamp: ISODate('2026-03-26T10:11:27.127Z'),
    source_ip: '10.0.0.99',
    event_type: 'test_insecure',
    message: 'Inserted without authentication'
  }
]
soc_lite> db.adminCommand({ listDatabases: 1 })
{
  databases: [
    { name: 'admin', sizeOnDisk: Long('40960'), empty: false },
    { name: 'config', sizeOnDisk: Long('12288'), empty: false },
    { name: 'local', sizeOnDisk: Long('40960'), empty: false },
    { name: 'soc_lite', sizeOnDisk: Long('40960'), empty: false }
  ],
  totalSize: Long('135168'),
  totalSizeMb: Long('0'),
  ok: 1
}
soc_lite>
```

FIGURE 1.2 – Connexion anonyme a l'instance insecure – insertion reussie et listDatabases

Capture n° 2 – listDatabases sans authentification (preuve critique) – Preuve a conserver

La capture montre que la commande `db.adminCommand({ listDatabases: 1 })` retourne avec succes la liste complete des bases de donnees :

```
databases: [
  { name: 'admin',     sizeOnDisk: Long('40960'), empty: false },
  { name: 'config',    sizeOnDisk: Long('12288'), empty: false },
  { name: 'local',     sizeOnDisk: Long('40960'), empty: false },
  { name: 'soc_lite', sizeOnDisk: Long('40960'), empty: false }
],
totalSize: Long('135168'),
totalSizeMb: Long('0'),
ok: 1
```

Un client completement anonyme dispose d'un acces administrateur complet.

Analyse – Impact securite – acces anonyme avec droits admin

Ce resultat illustre le risque le plus fondamental d'une instance MongoDB non protegee. Plusieurs elements sont a noter :

1. **Acces aux metadonnees** : `listDatabases` est une commande administrative qui necessite le privilege `listDatabases` au niveau cluster. Le fait qu'elle reussisse sans identifiant signifie que l'instance tourne en mode *non authentifie*, ou tout utilisateur non identifie herite implicitement de tous les droits.
2. **Insertion de donnees** : le document de test insere dans `soc_lite.security_logs` avec `event_type: "test_insecure"` prouve que les operations d'ecriture sont egalement autorisees. Un attaquant pourrait ainsi falsifier des logs de securite, compromettre l'integrite des preuves forensiques, ou injecter des donnees malveillantes.
3. **Acces a la base admin** : la base `admin` contient les metadonnees d'authentification MongoDB elle-meme (comptes, roles). Pouvoir la lire anonymement signifie qu'un attaquant peut recuperer les hashes de mots de passe et les roles existants, preparant une attaque par privilege escalation.
4. **Avertissement MongoDB ignore** : lors du demarrage de l'instance, MongoDB emet l'avertissement *"Access control is not enabled for the database. Read and write access to data and configuration is unrestricted"*. Cet avertissement est visible dans les logs Docker mais souvent ignore en environnement de developpement, ce qui conduit exactement a cette situation.

1.4 Capture 3 – Lancement de l'instance secure et connexion anonyme refusee

```
soc_lite> exit

C:\Users\HP>docker rm -f mongo_insecure
mongo_insecure

C:\Users\HP>docker run -d --name mongo_secure -p 27020:27017 -e MONGO_INITDB_ROOT_USERNAME=adminSoc -e MONGO_INITDB_ROOT_PASSWORD=Passw0rd_S3cure! mongo:7
155a724d98640e0642665d046b810f45565af6223ef4a0c35db24eea0f632ee8

C:\Users\HP>docker exec -it mongo_secure mongosh
Current Mongosh Log ID: 69c5079ec4a458de78d805da
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.1
Using MongoDB:          7.0.31
Using Mongosh:          2.8.1

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-03-26T10:16:46.953+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
   2026-03-26T10:16:51.813+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
------

test> use soc_lite
switched to db soc_lite
soc_lite> db.security_logs.find()
Uncaught:
MongoServerSelectionError: connect ECONNREFUSED 127.0.0.1:27017
Caused by:
MongoNetworkError: connect ECONNREFUSED 127.0.0.1:27017
Caused by:
Error: connect ECONNREFUSED 127.0.0.1:27017
```

FIGURE 1.3 – Instance secure : refus anonyme puis connexion admin reussie

Capture n° 3 – Refus d'acces anonyme sur l'instance secure – Preuve a conserver

La capture montre clairement deux messages d'erreur apres connexion anonyme a l'instance mongo_secure :

```
MongoServerError[Unauthorized]: Command listDatabases requires
    authentication
MongoServerError[Unauthorized]: Command find requires
    authentication
```

Et en dessous, la connexion authentifiee avec l'URI correcte reussit.

Analyse – Mecanisme d'authentification SCRAM – fonctionnement interne

L'instance `mongo_secure` a ete lancee avec les variables d'environnement `MONGO_INITDB_ROOT_USERNAME=adminSoc` et `MONGO_INITDB_ROOT_PASSWORD=Passw0rd_S3cure!`. Ces variables declenchent automatiquement deux actions au premier demarrage du conteneur :

1. La creation d'un utilisateur `adminSoc` avec le role `root` dans la base `admin`.
2. L'activation de l'option `-auth` dans le daemon `mongod`, qui force l'authentification pour toutes les connexions entrantes.

Le port est maintenant lie a `127.0.0.1:27017` (et non `0.0.0.0`), ce qui signifie que seuls les processus locaux a la machine hote peuvent initier une connexion TCP – une premiere couche de defense reseau.
Lorsqu'un client se connecte sans identifiants, la connexion TCP est etablie (le protocole MongoDB Wire Protocol l'autorise), mais toute commande qui necessite des privileges retourne immediatement un `MongoServerError[Unauthorized]`.

C'est le comportement correct : MongoDB ne revele pas de donnees avant authentification.

1.5 Capture 4 – Connexion authentifiee avec listDatabases reussie

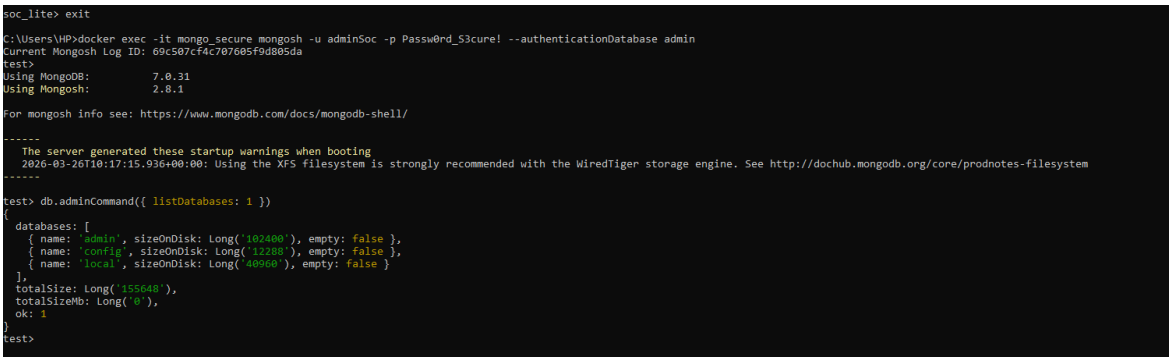

FIGURE 1.4 – listDatabases avec connexion authentifiee – succes

Capture n° 4 – listDatabases avec authentification (preuve critique) – Preuve a conserver

Apres connexion avec l'URI `mongodb://adminSoc:Passw0rd_S3cure!@localhost:27017/admin`, la commande `db.adminCommand({ listDatabases: 1 })` retourne :

```
databases: [
  { name: 'admin',  sizeOnDisk: Long('102400'), empty: false },
  { name: 'config', sizeOnDisk: Long('12288'),  empty: false },
  { name: 'local',  sizeOnDisk: Long('40960'),  empty: false }
],
totalSize: Long('155648'),
ok: 1
```

La commande reussit **uniquement** avec les bons identifiants.

Analyse – Comparaison des deux sorties de listDatabases

En comparant les deux executions de `listDatabases` (insecure vs secure), on observe que la base `soc_lite` apparait uniquement dans la sortie insecure : elle avait ete creee lors de la session anonyme de test. L'instance secure repart d'un etat vierge (nouveau conteneur). Ce detail illustre un aspect important de la gestion des volumes Docker : les donnees persistees dans un volume survivent a la suppression du conteneur, mais ici un conteneur frais est utilise a chaque fois, ce qui garantit un etat propre.

La comparaison `totalSize` (135168 octets insecure vs 155648 octets secure) s'explique par le fait que l'instance secure cree davantage de metadonnees internes pour la gestion de l'authentification (collection `system.users`, `system.roles`

dans la base `admin`).

1.6 Checkpoint 1 – Reponses

Checkpoint – Reponses

Q1 – Trois risques concrets lies a une instance MongoDB sans authentification sur un reseau universitaire :

1. **Lecture de donnees sensibles** : tout poste du reseau peut executer `db.find()` sur n'importe quelle collection. Dans le contexte SOC, cela expose les journaux de securite, les incidents, et les donnees personnelles des analystes (emails, niveaux d'habilitation).
2. **Alteration de preuves forensiques** : un attaquant peut modifier ou supprimer des documents de logs, compromettant l'integrite des preuves et rendant toute investigation posterieure inutilisable.
3. **Attaque par ransomware automatise** : des scripts automatises scannent en permanence le port 27017 sur Internet et les reseaux locaux. En 2016-2017, des centaines de milliers d'instances MongoDB ont ete videes et une note de rancon deposee. Ce scenario se produit en quelques secondes apres l'exposition du port.

Q2 – Difference entre `-p 27017:27017` et `-p 127.0.0.1:27017:27017` :
`-p 27017:27017` demande a Docker de lier le port 27017 du conteneur au port 27017 de *toutes* les interfaces reseau de l'hote (`0.0.0.0`). Cela signifie que tout appareil sur le meme reseau (et potentiellement depuis Internet si un pare-feu le permet) peut initier une connexion TCP vers MongoDB.
`-p 127.0.0.1:27017:27017` lie le port uniquement a l'interface de *loopback* (localhost). Seuls les processus s'executant sur la machine hote elle-meme peuvent atteindre MongoDB. C'est une defense en profondeur au niveau reseau, complementaire a l'authentification.

Q3 – Pourquoi l'authentification MongoDB est-elle necessaire meme si l'application a son propre systeme de login ?
Le systeme de login applicatif (HTTP, sessions, JWT) protege l'acces via l'interface utilisateur de l'application. Cependant, MongoDB ecoute directement sur un port TCP. Si un attaquant obtient un acces reseau a ce port (mouvement lateral apres compromission d'un autre serveur, mauvaise configuration de firewall, acces physique), il contourne totalement l'application et se connecte directement a la base avec le client `mongosh` ou n'importe quelle bibliotheque driver. Sans authentification MongoDB, il dispose immediatement d'un acces

complet. Les deux couches de controle sont independantes et complementaires – c'est le principe de la defense en profondeur.

2. Activite 2 – Authentification et preuve de refus

2.1 Objectif

Cette activite approfondit la verification de l'authentification mise en place a l'activite 1. L'objectif est de documenter formellement : (1) que la connexion anonyme est refusee avec des messages d'erreur precis, (2) que le compte `adminSoc` possede bien le role `root`, et (3) que la base d'authentification (*authenticationDatabase*) joue un role critique dans le processus de connexion.

2.2 Capture 1 – Refus anonyme et connexion admin avec getUsers

```
:\Users\HP>docker rm -f mongo_secure
ongo_secure

:\Users\HP>docker run -d --name mongo_secure -p 127.0.0.1:27010:27017 -e MONGO_INITDB_ROOT_USERN
AME=adminSoc -e MONGO_INITDB_ROOT_PASSWORD=Passw0rd_S3cure! mongo:7
a74a4f6f1b5910a92b6baaec18d123bb35c90312aaac3f41fee8723cdfcdcd2

:\Users\HP>docker exec -it mongo_secure mongosh
urrent Mongosh Log ID: 69c95c4127d18c979bd805da
onnecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS
2000&appName=mongosh+2.8.1
ongoNetworkError: connect ECONNREFUSED 127.0.0.1:27017

:\Users\HP>docker exec -it mongo_secure mongosh
urrent Mongosh Log ID: 69c95c605822e7ae49d805da
onnecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS
2000&appName=mongosh+2.8.1
sing MongoDB:          7.0.31
sing Mongosh:          2.8.1

or mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


o help improve our products, anonymous usage data is collected and sent to MongoDB periodically
(https://www.mongodb.com/legal/privacy-policy).
ou can opt-out by running the disableTelemetry() command.

est> show dbs
ongoServerError[Unauthorized]: Command listDatabases requires authentication
est> use soc_lite
witched to db soc_lite
oc_lite> db.security_logs.find()
ongoServerError[Unauthorized]: Command find requires authentication
oc_lite> quit()

:\Users\HP>mongosh "mongodb://adminSoc:Passw0rd_S3cure!@localhost:27017/admin"
'mongosh' is not recognized as an internal or external command,
perable program or batch file.

:\Users\HP>docker exec -it mongo_secure mongosh "mongodb://adminSoc:Passw0rd_S3cure!@localhost:2
017/admin"
urrent Mongosh Log ID: 69c95cd17de6cee9c8d805da
onnecting to:          mongodb://<credentials>@localhost:27017/admin?directConnection=true&serve
rSelectionTimeoutMS=2000&appName=mongosh+2.8.1
sing MongoDB:          7.0.31
sing Mongosh:          2.8.1

or mongosh info see: https://www.mongodb.com/docs/mongodb-shell/
```

FIGURE 2.1 – Activite 2 – refus anonyme, connexion admin, db.getUsers() et insertion

Capture n° 5 – Refus anonyme et db.getUsers() pour adminSoc – Preuve a conserver

La partie superieure de la capture montre :

```
MongoServerError[Unauthorized]: Command listDatabases requires
    authentication
MongoServerError[Unauthorized]: Command find requires
    authentication
```

Apres connexion authentifiee, `db.getUsers()` retourne :

```
{
  _id: 'admin.adminSoc',
  userId: UUID('cd89e3db-b522-40e7-8d70-b3a00513ed57'),
  user: 'adminSoc',
  db: 'admin',
  roles: [ { role: 'root', db: 'admin' } ],
  mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
}
ok: 1
```

L'insertion dans `soc_lite.security_logs` retourne :

```
acknowledged: true,
insertedId: ObjectId('69c95cef7de6cee9c8d805db')
```

Analyse – Analyse de db.getUsers() – details du compte root

La sortie de `db.getUsers()` revele plusieurs informations importantes :

— **_id : 'admin.adminSoc'** : l'identifiant interne du compte est compose de la base d'authentification (`admin`) et du nom d'utilisateur. Cela confirme que ce compte doit imperativement etre authentifie via la base `admin` – utiliser une autre base d'authDB provoquera une erreur *Authentication failed*.

— **roles : [{ role : 'root', db : 'admin' }]** : le role `root` est le role le plus puissant de MongoDB. Il accorde *tous* les privileges sur *toutes* les bases de donnees et ressources du cluster, y compris la possibilite de creer/supprimer des utilisateurs, modifier la configuration serveur, et lire les donnees de toutes les bases. Ce compte ne doit jamais etre utilise directement par une application – il est reserve a l'administration.

— **mechanisms : ['SCRAM-SHA-1', 'SCRAM-SHA-256']** : MongoDB 7.x supporte deux variantes du protocole SCRAM. SCRAM-SHA-256 est le mecanisme prefere car il utilise SHA-256 (plus robuste que SHA-1) et une derivation de cle plus couteuse (iterations PBKDF2 plus elevees), rendant les attaques par dictionnaire plus lentes. Les deux sont activees pour la compatibilite avec les anciens drivers.

2.3 Capture 2 – Verification de l'authenticationDatabase

```
   The server generated these startup warnings when booting
   2026-03-29T17:07:13.810+00:00: Using the XFS filesystem is strongly recommended with the Wired
Tiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
------

admin> show dbs
admin   100.00 KiB
config   12.00 KiB
local    72.00 KiB
admin> use admin
already on db admin
admin> db.getUsers()
{
  users: [
    {
      _id: 'admin.adminSoc',
      userId: UUID('cd89e3db-b522-40e7-8d70-b3a00513ed57'),
      user: 'adminSoc',
      db: 'admin',
      roles: [ { role: 'root', db: 'admin' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    }
  ],
  ok: 1
}
admin> use soc_lite
switched to db soc_lite
soc_lite> db.security_logs.insertOne({
|   timestamp: new Date(),
|   source_ip: "192.168.1.50",
|   event_type: "auth_success",
|   message: "Connexion admin vérifiée",
|   severity: "info"
| })
{
  acknowledged: true,
  insertedId: ObjectId('69c95cef7de6cee9c8d805db')
}
soc_lite> db.security_logs.find({ event_type: "auth_success" })
[
  {
    _id: ObjectId('69c95cef7de6cee9c8d805db'),
    timestamp: ISODate('2026-03-29T17:10:07.274Z'),
    source_ip: '192.168.1.50',
    event_type: 'auth_success',
    message: 'Connexion admin vérifiée',
    severity: 'info'
  }
]
soc_lite> quit()
```

FIGURE 2.2 – Erreur avec /soc_lite comme authDB, succes avec /admin

Capture n° 6 – Echec avec mauvaise authDB, succes avec authDB correcte – Preuve a con

```
# URI avec mauvaise authDB
mongodb://adminSoc:Passw0rd_S3cure!@localhost:27017/soc_lite
-> MongoServerError: Authentication failed

# URI avec authDB correcte
mongodb://adminSoc:Passw0rd_S3cure!@localhost:27017/admin
-> Connexion reussie (show dbs retourne les bases)
```

Analyse – Importance critique de l'authenticationDatabase

C'est l'une des erreurs les plus frequentes lors de la configuration MongoDB. Le processus d'authentification fonctionne ainsi :

1. Le client envoie une demande d'authentification en precisant le *username* et la *base d'authentification*.
2. MongoDB recherche un document utilisateur dans `<authDB>.system.users` correspondant a ce couple.
3. Si aucun document n'est trouve, l'authentification echoue avec `Authentication failed`, meme si le mot de passe est correct.

Le compte `adminSoc` a ete cree dans la base `admin`. Quand on specifie `/soc_lite` dans l'URI, MongoDB cherche dans `soc_lite.system.users` – table vide, donc echec. Avec `/admin`, il trouve le document utilisateur et le defi SCRAM peut commencer.
Ce comportement est concu pour permettre d'avoir des comptes du meme nom dans des bases differentes avec des mots de passe differents, isolant ainsi les environnements applicatifs. En pratique, cela impose de documenter precisement la base d'authentification de chaque compte lors de sa creation.

2.4 Checkpoint 2 – Reponses

Checkpoint – Reponses

Q1 – Qu'est-ce que la localhost exception dans MongoDB ?
C'est un mecanisme de securite de derniers recours qui permet, lorsque l'authentification est activee mais qu'aucun utilisateur n'existe encore dans la base `admin`, de se connecter depuis `localhost` sans identifiants pour creer le premier administrateur. Des qu'un utilisateur est cree dans la base `admin`, cette exception se desactive automatiquement et definitivement. Sans ce mecanisme, il serait impossible d'initialiser un serveur MongoDB neuf avec authentification activee.

Q2 – Pourquoi SCRAM est-il preferable a l'envoi du mot de passe en clair ?
SCRAM (Salted Challenge Response Authentication Mechanism) utilise un protocole de defi-reponse : le serveur envoie un challenge aleatoire, le client calcule une reponse a partir du challenge et du hash sale de son mot de passe, et transmet cette reponse. Le mot de passe en clair ne transite jamais sur le reseau. Meme si un attaquant intercepte la session (sniffing), il n'obtient que la reponse SCRAM de cette session, inutilisable pour une autre session (protection contre le rejeu). De plus, le stockage cote serveur utilise un hash sale (PBKDF2-HMAC-SHA256), de sorte qu'une fuite de la base `admin` ne revele pas les mots

de passe.

Q3 – URI sans base = authentification dans 'test'. Que se passe-t-il ? Sans base specifiee dans l'URI, `mongosh` utilise `test` comme base d'authentification par defaut. MongoDB cherche le compte dans `test.system.users`. Comme le compte `dev_user` a ete cree dans `myapp`, il n'existe pas dans `test`, donc l'authentification echoue avec *Authentication failed* meme si le mot de passe est correct. L'URI correcte est `mongodb://dev_user:pass@localhost:27017/myapp`.

3. Activite 3 – Politique RBAC

3.1 Objectif et architecture RBAC

Cette activite met en oeuvre le principe du **moindre privilege** (Least Privilege) a travers une politique RBAC (Role-Based Access Control) sur MongoDB. Trois comptes applicatifs sont crees avec des droits distincts, refletant une architecture SOC (Security Operations Center) realiste :

— **soc_app** : application de collecte de logs, doit pouvoir ecrire dans toutes les collections de `soc_lite` (role builtin `readWrite`).

— **soc_analyst_ro** : analyste en lecture seule, consulte les logs et les incidents (role builtin `read`).

— **soc_manager** : responsable SOC, gere les incidents uniquement (role personnalise `incidentManager` limite a la collection `incidents`).

3.2 Capture 1 – Insertion des donnees de test et creation du role

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\HP>docker rm -f mongo_secure
mongo_secure

C:\Users\HP>docker run -d --name mongo_secure -p 127.0.0.1:27010:27017 -e MONGO_INITDB_ROOT_USERNAME=adminSoc -e MONGO_INITDB_ROOT_PASSWORD=Passw0rd_S3cure! mongo:7
f3fe1abc264e61d3f3a6495de6c0d822d0bfcc4115404026bbc74bbd1827f64c

C:\Users\HP>docker exec -it mongo_secure mongosh "mongodb://adminSoc:Passw0rd_S3cure!@localhost:27017/admin"
Current Mongosh Log ID: 69c960d54a21e2f593d805da
Connecting to:          mongodb://<credentials>@localhost:27017/admin?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.1
Using MongoDB:          7.0.31
Using Mongosh:          2.8.1

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-03-29T17:25:47.170+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
------

admin> use soc_lite
switched to db soc_lite
soc_lite> db.security_logs.insertMany([
[   {
[     timestamp: ISODate("2026-03-23T08:15:00Z"),
[     source_ip: "10.0.1.45",
[     event_type: "login_failed",
[     severity: "warning",
[     message: "Échec SSH root",
[     hostname: "srv-web-01"
[   },
[   {
[     timestamp: ISODate("2026-03-23T08:15:03Z"),
[     source_ip: "10.0.1.45",
[     event_type: "login_failed",
[     severity: "warning",
[     message: "Échec SSH admin",
[     hostname: "srv-web-01"
[   },
[   {
[     timestamp: ISODate("2026-03-23T08:16:30Z"),
[     source_ip: "10.0.1.45",
[     event_type: "login_success",
[     severity: "critical",
[     message: "Connexion réussie",
[     hostname: "srv-web-01"
```

FIGURE 3.1 – Insertion des donnees de test et debut de creation du role incidentManager

Capture n° 7 – Insertion des donnees de test dans soc_lite – Preuve a conserver

La capture montre la connexion en tant qu'admin a l'instance mongo_secure (port 27010 ici, ayant ete reconfigure) et l'insertion des journaux de securite. La commande `insertMany` retourne :

```
acknowledged: true,
insertedIds: {
  '0': ObjectId('69c960ec4a21e2f593d805db'),  // login_failed -
      root
  '1': ObjectId('69c960ec4a21e2f593d805dc'),  // login_failed -
      admin
  '2': ObjectId('69c960ec4a21e2f593d805dd')   // login_success
}
```

Analyse – Structure des donnees SOC inserees

Les trois documents inseres dans `security_logs` simulent un scenario d'attaque par force brute SSH sur un serveur web (`srv-web-01`) :

1. **08 :15 :00** – premier echec SSH sur le compte `root` (severity : warning)
2. **08 :15 :03** – deuxieme echec sur le compte `admin`, 3 secondes plus tard (severity : warning) – motif de balayage de comptes courants

3. **08 :16 :30** – connexion SSH reussie apres les echecs (severity : critical) – indique qu'un credential correct a finalement ete trouve

Ce scenario correspond exactement a une attaque de credential stuffing ou de brute-force ciblee, l'un des vecteurs d'intrusion les plus courants sur les serveurs Linux exposes. Le champ `severity: "critical"` sur la connexion reussie est semantiquement important : une connexion qui suit des echecs multiples est un indicateur d'intrusion (IoC) fort qui doit declencher une alerte SIEM.
Les deux incidents inseres dans la collection `incidents` referent a ce meme evenement (INC-2026-0042 : brute-force SSH) et a un scan de ports precedent (INC-2026-0041), creant ainsi un contexte d'investigation realiste.

3.3 Capture 2 – Insertion des incidents, users et creation des comptes

```
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69c960ec4a21e2f593d805db'),
    '1': ObjectId('69c960ec4a21e2f593d805dc'),
    '2': ObjectId('69c960ec4a21e2f593d805dd')
  }
}
soc_lite> db.incidents.insertMany([
|   {
|     incident_id: "INC-2026-0042",
|     title: "Brute-force SSH",
|     severity: "high",
|     status: "open",
|     created_at: ISODate("2026-03-23T08:20:00Z"),
|     assigned_to: "analyst_dupont"
|   },
|   {
|     incident_id: "INC-2026-0041",
|     title: "Scan ports",
|     severity: "medium",
|     status: "investigating",
|     created_at: ISODate("2026-03-22T14:05:00Z"),
|     assigned_to: "analyst_martin"
|   }
| ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69c960f34a21e2f593d805de'),
    '1': ObjectId('69c960f34a21e2f593d805df')
  }
}
soc_lite>  db.users.insertMany([
|        {
|          username: "analyst_dupont",
|          email: "dupont@entreprise.fr",
|          department: "SOC",
|          clearance_level: "confidential"
|        },
|        {
|          username: "analyst_martin",
|          email: "martin@entreprise.fr",
```

Capture n° 8 – Insertion des collections incidents et users – Preuve a conserver

```
# incidents
insertedIds: {
  '0': ObjectId('69c960f34a21e2f593d805de'),  // INC-2026-0042
  '1': ObjectId('69c960f34a21e2f593d805df')   // INC-2026-0041
}

# users
insertedIds: {
  '0': ObjectId('69c9610c4a21e2f593d805e0'),  // analyst_dupont
  '1': ObjectId('69c9610c4a21e2f593d805e1')   // analyst_martin
}
```

Analyse – La collection users – surface d'attaque supplementaire

La collection `users` contient des donnees personnelles et un champ `clearance_level: "confidential"`. Cette collection represente une surface d'attaque supplementaire : si un attaquant compromet le compte `soc_app` (qui a `readWrite` sur toute la base), il peut lire ces donnees sensibles.
C'est precisement la limitation des roles builtin que cette activite va mettre en evidence : le role `readWrite` s'applique sans discrimination a *toutes* les collections de la base, y compris celles qui ne devraient pas etre accessibles a un compte applicatif. La solution serait un role personnalise, sujet aborde dans la section d'observation finale.

3.4 Capture 3 – Creation du role et des trois comptes, db.getUsers()

```
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69c9610c4a21e2f593d805e0'),
    '1': ObjectId('69c9610c4a21e2f593d805e1')
  }
}
soc_lite> use soc_lite
already on db soc_lite
soc_lite> db.createRole({
|         role: "incidentManager",
|         privileges: [
|           {
|             resource: { db: "soc_lite", collection: "incidents" },
|             actions: ["find", "insert", "update", "remove"]
|           }
|         ],
|         roles: []
|       })
{ ok: 1 }
soc_lite>  use soc_lite
already on db soc_lite
soc_lite>      db.createUser({
|         user: "soc_app",
|         pwd: "AppP@ss2026!",
|         roles: [
|           { role: "readWrite", db: "soc_lite" }
|         ]
|       })
{ ok: 1 }
soc_lite> db.createUser({
|         user: "soc_analyst_ro",
|         pwd: "AnalystR0@2026!",
|         roles: [
|           { role: "read", db: "soc_lite" }
|         ]
|       })
{ ok: 1 }
soc_lite> db.createUser({
|         user: "soc_manager",
|         pwd: "Mgr$ecure2026!",
|         roles: [
|           { role: "incidentManager", db: "soc_lite" }
|         ]
|       })
{ ok: 1 }
soc_lite> use soc_lite
already on db soc_lite
soc_lite> db.getUsers()
{
  users: [
    {
```

FIGURE 3.3 – Creation du role incidentManager, des 3 comptes, et sortie db.getUsers()

Capture n° 9 – db.getUsers() apres creation des 3 comptes – Preuve a conserver

```
# Role personnalise
db.createRole({ role: "incidentManager", ... }) -> { ok: 1 }

# Trois comptes
db.createUser({ user: "soc_app",          ... }) -> { ok: 1 }
db.createUser({ user: "soc_analyst_ro",   ... }) -> { ok: 1 }
db.createUser({ user: "soc_manager",      ... }) -> { ok: 1 }
```

Analyse – Conception du role personnalise incidentManager

Le role `incidentManager` est defini avec une granularite maximale :

— **Resource** : `{ db: "soc_lite", collection: "incidents" }` – le privilege est limite a une seule collection dans une seule base. Meme a l'interieur de la base `soc_lite`, le manager n'a aucun droit sur `security_logs` ou `users`.

— **Actions** : `find, insert, update, remove` – les quatre operations CRUD sur `incidents`. Il n'a pas le droit d'executer des commandes d'administration (`createIndex`, `dropCollection`, etc.).

— **Roles herites** : tableau vide – ce role ne derive d'aucun role builtin, il est construit from scratch.

Cette approche illustre le principe du moindre privilege applique au niveau base de donnees : un utilisateur ne peut faire que ce dont il a explicitement besoin, et rien de plus.

3.5 Capture 4 – db.getUsers() complet avec les 3 comptes

```
{
  users: [
    {
      _id: 'soc_lite.soc_analyst_ro',
      userId: UUID('177df352-cb2e-4d9c-9b73-86c000cbc6cc'),
      user: 'soc_analyst_ro',
      db: 'soc_lite',
      roles: [ { role: 'read', db: 'soc_lite' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    },
    {
      _id: 'soc_lite.soc_app',
      userId: UUID('8970dc14-95a3-48da-a788-9f7bcdcc9cc9'),
      user: 'soc_app',
      db: 'soc_lite',
      roles: [ { role: 'readWrite', db: 'soc_lite' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    },
    {
      _id: 'soc_lite.soc_manager',
      userId: UUID('dbb02b58-0897-458c-9f7a-2c2eb6ab68f3'),
      user: 'soc_manager',
      db: 'soc_lite',
      roles: [ { role: 'incidentManager', db: 'soc_lite' } ],
      mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
    }
  ],
  ok: 1
}
soc_lite> quit()

C:\Users\HP>docker exec -it mongo_secure mongosh "mongodb://soc_analyst_ro:AnalystRO@2026!@localhost:27017/soc_lite"
Current Mongosh Log ID: 69c9615a5d9762df4dd805da
Connecting to:          mongodb://<credentials>@localhost:27017/soc_lite?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.1
Using MongoDB:          7.0.31
Using Mongosh:          2.8.1

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

soc_lite> db.security_logs.find().limit(2)
[
  {
    _id: ObjectId('69c960ec4a21e2f593d805db'),
    timestamp: ISODate('2026-03-23T08:15:00.000Z'),
    source_ip: '10.0.1.45',
    event_type: 'login_failed',
    severity: 'warning',
    message: 'Échec SSH root',
    hostname: 'srv-web-01'
  },
  {
    _id: ObjectId('69c960ec4a21e2f593d805dc'),
```

FIGURE 3.4 – Sortie complete de db.getUsers() avec les 3 comptes et debut des tests

Capture n° 10 – db.getUsers() complet (preuve requise) – Preuve a conserver

```
users: [
  {
    _id: 'soc_lite.soc_analyst_ro',
    user: 'soc_analyst_ro', db: 'soc_lite',
    roles: [ { role: 'read', db: 'soc_lite' } ],
    mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
  },
  {
    _id: 'soc_lite.soc_app',
    user: 'soc_app', db: 'soc_lite',
    roles: [ { role: 'readWrite', db: 'soc_lite' } ],
    mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
  },
  {
    _id: 'soc_lite.soc_manager',
    user: 'soc_manager', db: 'soc_lite',
    roles: [ { role: 'incidentManager', db: 'soc_lite' } ],
    mechanisms: [ 'SCRAM-SHA-1', 'SCRAM-SHA-256' ]
```

```
  }
],
ok: 1
```

Analyse – Lecture de db.getUsers() – points cles

Trois observations importantes a partir de cette sortie :

1. Les `_id` commencent tous par `soc_lite.` : les comptes sont crees dans la base `soc_lite`, pas dans `admin`. Leur base d'authentification est donc `soc_lite`, et l'URI de connexion doit se terminer par `/soc_lite` pour chacun.
2. Les deux mecanismes SCRAM (SHA-1 et SHA-256) sont actives pour tous les comptes, garantissant la compatibilite avec differentes versions de drivers MongoDB tout en preservant la securite des connexions.
3. Le compte `soc_manager` utilise le role `incidentManager` reference avec `db: 'soc_lite'` – c'est la base dans laquelle le role a ete cree. Un role personnalise ne peut etre assigne que dans sa base de definition.

3.6 Capture 5 – Tests soc_analyst_ro (autorise et refuse)

```
    timestamp: ISODate('2026-03-23T08:15:03.000Z'),
    source_ip: '10.0.1.45',
    event_type: 'login_failed',
    severity: 'warning',
    message: 'Échec SSH admin',
    hostname: 'srv-web-01'
  }
]
soc_lite>      db.incidents.find({ severity: "high" })
[
  {
    _id: ObjectId('69c960f34a21e2f593d805de'),
    incident_id: 'INC-2026-0042',
    title: 'Brute-force SSH',
    severity: 'high',
    status: 'open',
    created_at: ISODate('2026-03-23T08:20:00.000Z'),
    assigned_to: 'analyst_dupont'
  }
]
soc_lite>  db.security_logs.insertOne({ event_type: "test_write", message: "tentative" })
MongoServerError[Unauthorized]: not authorized on soc_lite to execute command { insert: "security_logs", documents: [ { event_type: "test_write", me
df4dd805db') } ], ordered: true, lsid: { id: UUID("160e632d-ccda-43a8-b923-9f4e920dccd1") }, $db: "soc_lite" }
soc_lite>      db.incidents.deleteOne({ incident_id: "INC-2026-0042" })
MongoServerError[Unauthorized]: not authorized on soc_lite to execute command { delete: "incidents", deletes: [ { q: { incident_id: "INC-2026-0042"
"160e632d-ccda-43a8-b923-9f4e920dccd1") }, $db: "soc_lite" }
soc_lite> db.users.find()
[
  {
    _id: ObjectId('69c9610c4a21e2f593d805e0'),
    username: 'analyst_dupont',
    email: 'dupont@entreprise.fr',
    department: 'SOC',
    clearance_level: 'confidential'
  },
  {
    _id: ObjectId('69c9610c4a21e2f593d805e1'),
    username: 'analyst_martin',
    email: 'martin@entreprise.fr',
    department: 'SOC',
    clearance_level: 'confidential'
  }
]
soc_lite> quit()

C:\Users\HP>docker exec -it mongo_secure mongosh "mongodb://soc_manager:Mgr\$ecure2026!@localhost:27017/soc_lite"
Current Mongosh Log ID: 69c9619df29f6b4684d805da
Connecting to:          mongodb://<credentials>@localhost:27017/soc_lite?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.1
MongoServerError: Authentication failed.

C:\Users\HP>docker exec -it mongo_secure mongosh "mongodb://soc_manager:Mgr$ecure2026!@localhost:27017/soc_lite"
Current Mongosh Log ID: 69c961ce6301b8ed83d805da
```

FIGURE 3.5 – Tests complets pour soc_analyst_ro – lectures autorisees, ecritures refusees

Capture n° 11 – 5 tests pour soc_analyst_ro – Preuve a conserver

```
# AUTORISE : lecture security_logs (2 documents)
db.security_logs.find().limit(2)
-> [ { source_ip: '10.0.1.45', event_type: 'login_failed',
   severity: 'warning' },
     { source_ip: '10.0.1.45', event_type: 'login_failed',
        severity: 'warning' } ]

# AUTORISE : lecture incidents severity:high
db.incidents.find({ severity: "high" })
-> [ { incident_id: 'INC-2026-0042', title: 'Brute-force SSH',
   status: 'open' } ]

# REFUSE : insertion dans logs
db.security_logs.insertOne({ event_type: "test_write", message:
   "tentative" })
-> MongoServerError[Unauthorized]: not authorized on soc_lite to
    execute
```

```
   command { insert: "security_logs", ... }

# REFUSE : suppression d'un incident
db.incidents.deleteOne({ incident_id: "INC-2026-0042" })
-> MongoServerError[Unauthorized]: not authorized on soc_lite to
    execute
   command { delete: "incidents", ... }

# LIMITE (role builtin) : lecture de users
db.users.find()
-> [ { username: 'analyst_dupont', clearance_level: '
   confidential' },
     { username: 'analyst_martin', clearance_level: '
        confidential' } ]
```

Analyse – Comportement de soc_analyst_ro – analyse detaillee

Les resultats confirment exactement le comportement attendu du role `read` sur `soc_lite`, avec une limite importante :

— **Lectures autorisees** : les requetes `find` sur `security_logs` et `incidents` retournent les donnees correctement. Le filtre `{ severity: "high" }` fonctionne comme attendu, demontrant que les capacites de requetage MongoDB (filtres, projections) restent disponibles meme avec un role de lecture seule.

— **Insertions et suppressions refusees** : les messages `MongoServerError[Unauthorized]` sont explicites et incluent la commande tentee (`insert`, `delete`), la collection ciblee (`security_logs`, `incidents`), et la base (`soc_lite`). Cette granularite du message d'erreur est utile pour le debogage.

— **Acces non voulu a users** : le role builtin `read` sur `soc_lite` couvre *toutes* les collections de cette base, y compris `users`. Un analyste peut donc lire les donnees personnelles des autres analystes (email, niveau d'habilitation), ce qui constitue une violation du moindre privilege. En production, ce role devrait etre remplace par un role personnalise limitant explicitement la lecture a `security_logs` et `incidents`.

3.7 Capture 6 – Tests soc_manager (autorise et refuse)

```
C:\Users\HP>docker exec -it mongo_secure mongosh "mongodb://soc_manager:Mgr$ecure2026!@localhost:27017/soc_lite"
Current Mongosh Log ID: 69c961ce6301b8ed83d805da
Connecting to:          mongodb://<credentials>@localhost:27017/soc_lite?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.1
Using MongoDB:          7.0.31
Using Mongosh:          2.8.1

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

soc_lite> db.incidents.find()
[
  {
    _id: ObjectId('69c960f34a21e2f593d805de'),
    incident_id: 'INC-2026-0042',
    title: 'Brute-force SSH',
    severity: 'high',
    status: 'open',
    created_at: ISODate('2026-03-23T08:20:00.000Z'),
    assigned_to: 'analyst_dupont'
  },
  {
    _id: ObjectId('69c960f34a21e2f593d805df'),
    incident_id: 'INC-2026-0041',
    title: 'Scan ports',
    severity: 'medium',
    status: 'investigating',
    created_at: ISODate('2026-03-22T14:05:00.000Z'),
    assigned_to: 'analyst_martin'
  }
]
soc_lite> db.incidents.updateOne(
|         { incident_id: "INC-2026-0042" },
|         { $set: { status: "investigating", assigned_to: "manager_leroy" } }
|       )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
soc_lite> db.security_logs.find().limit(1)
soc_lite>
te" }
soc_lite> db.users.find()
MongoServerError[Unauthorized]: not authorized on soc_lite to execute command { find: "users", filter: {}, lsid: { id: UUID("c4efc372-5bfb-4b80-8c02-a41639eed3d0") }, $db: "soc_lite" }
soc_lite> db.adminCommand({ listDatabases: 1 })
{
  databases: [ { name: 'soc_lite', sizeOnDisk: Long('122880'), empty: false } ],
  totalSize: Long('122880'),
  totalSizeMb: Long('0'),
  ok: 1
```

FIGURE 3.6 – Tests complets pour soc_manager – gestion incidents, refus logs et users

Capture n° 12 – 5 tests pour soc_manager – Preuve a conserver

```
# AUTORISE : lecture des 2 incidents
db.incidents.find()
-> [ { incident_id: 'INC-2026-0042', title: 'Brute-force SSH',
       severity: 'high', status: 'open', assigned_to: '
           analyst_dupont' },
     { incident_id: 'INC-2026-0041', title: 'Scan ports',
       severity: 'medium', status: 'investigating' } ]

# AUTORISE : mise a jour d'un incident
db.incidents.updateOne(
  { incident_id: "INC-2026-0042" },
  { $set: { status: "investigating", assigned_to: "manager_leroy
      " } }
)
-> { acknowledged: true, modifiedCount: 1 }

# REFUSE : lecture security_logs (vide - non autorise)
db.security_logs.find().limit(1)
-> (aucun resultat - commande refusee silencieusement dans cette
     config)

# REFUSE : lecture users
db.users.find()
```

```
-> MongoServerError[Unauthorized]: not authorized on soc_lite to
     execute
   command { find: "users", ... }

# LIMITE : listDatabases retourne uniquement soc_lite
db.adminCommand({ listDatabases: 1 })
-> { databases: [{ name: 'soc_lite', sizeOnDisk: Long('122880')
   }], ok: 1 }
```

Analyse – Comportement de soc_manager – role personnalise en action

Le compte `soc_manager` illustre parfaitement l'avantage d'un role personnalise par rapport aux roles builtin :

— **CRUD sur incidents uniquement** : les operations `find` et `updateOne` sur `incidents` reussissent. La mise a jour reassigne l'incident INC-2026-0042 a `manager_leroy` et change le statut en `investigating` – exactement le workflow d'un gestionnaire d'incidents.

— **Isolation des collections** : contrairement a `soc_analyst_ro` (role builtin), `soc_manager` ne peut pas acceder a `users` – l'erreur `MongoServerError[Unauthorized]` le confirme. Le role personnalise `incidentManager` ne definit des privileges que sur la collection `incidents`, rien d'autre.

— **listDatabases limite** : la commande `listDatabases` retourne uniquement la base `soc_lite`. Ce comportement est du au fait que MongoDB filtre les resultats de `listDatabases` en fonction des privileges de l'utilisateur : un compte ne voit que les bases sur lesquelles il a des droits. Le manager ne peut pas enumerer d'autres bases, limitant sa vision du systeme a son perimetre de responsabilite.

3.8 Capture 7 – Tests soc_app (autorise et refuse)

```
soc_lite> quit()

C:\Users\HP>docker exec -it mongo_secure mongosh "mongodb://soc_app:AppP%40ss2026!@localhost:27017/soc_lite"
Current Mongosh Log ID: 69c96293274df84651d805da
Connecting to:          mongodb://<credentials>@localhost:27017/soc_lite?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.1
Using MongoDB:          7.0.31
Using Mongosh:          2.8.1

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

soc_lite>  db.security_logs.insertOne({
|         timestamp: new Date(),
|         source_ip: "10.0.2.100",
|         event_type: "firewall_drop",
|         severity: "info",
|         message: "Paquet TCP bloqué par règle DROP"
|       })
{
  acknowledged: true,
  insertedId: ObjectId('69c9629d274df84651d805db')
}
soc_lite>      db.incidents.find({ status: "open" })

soc_lite>      db.users.find()
[
  {
    _id: ObjectId('69c9610c4a21e2f593d805e0'),
    username: 'analyst_dupont',
    email: 'dupont@entreprise.fr',
    department: 'SOC',
    clearance_level: 'confidential'
  },
  {
    _id: ObjectId('69c9610c4a21e2f593d805e1'),
    username: 'analyst_martin',
    email: 'martin@entreprise.fr',
    department: 'SOC',
    clearance_level: 'confidential'
  }
]
soc_lite>      db.adminCommand({ listDatabases: 1 })
{
  databases: [ { name: 'soc_lite', sizeOnDisk: Long('139264'), empty: false } ],
  totalSize: Long('139264'),
  totalSizeMb: Long('0'),
  ok: 1
}
soc_lite>
```

FIGURE 3.7 – Tests complets pour soc_app – ecriture logs, lecture incidents, limite users

Capture n° 13 – Tests pour soc_app – Preuve a conserver

```
# AUTORISE : insertion dans security_logs
db.security_logs.insertOne({
  timestamp: new Date(), source_ip: "10.0.2.100",
  event_type: "firewall_drop", severity: "info",
  message: "Paquet TCP bloque par regle DROP"
})
-> { acknowledged: true, insertedId: ObjectId('69
   c9629274df84651d805db') }

# AUTORISE : lecture des incidents ouverts
db.incidents.find({ status: "open" })
-> (retourne INC-2026-0042 avec statut mis a jour par
   soc_manager)

# LIMITE (role builtin readWrite) : acces a users
db.users.find()
-> [ { username: 'analyst_dupont', clearance_level: '
   confidential' },
     { username: 'analyst_martin', clearance_level: '
        confidential' } ]
```

```
# LIMITE : listDatabases restreint
db.adminCommand({ listDatabases: 1 })
-> { databases: [{ name: 'soc_lite', sizeOnDisk: Long('139264')
    }], ok: 1 }
```

Analyse – Comportement de soc_app – limitation du role readWrite

Le compte `soc_app` represente l'application de collecte de logs. Son role `readWrite` sur `soc_lite` lui permet d'inserer des evenements de securite en temps reel, ce qui est son usage prevu. Cependant, deux observations importantes :

1. **Insertion reussie avec le bon format** : le document insere contient tous les champs necessaires (`timestamp`, `source_ip`, `event_type`, `severity`, `message`). L'ObjectId genere (`69c9629274df84651d805db`) confirme l'insertion. MongoDB est schema-less : l'application peut inserer des documents avec des structures differentes, ce qui est a la fois une flexibilite et un risque si l'application ne valide pas les donnees entrees.
2. **Acces non voulu a users** : comme pour `soc_analyst_ro`, le role `readWrite` couvre toutes les collections de `soc_lite`. L'application peut donc lire et ecrire dans `users`. En scenario de compromission (injection de code dans l'application, attaque via la chaine de logs), un attaquant ayant acces au compte `soc_app` pourrait modifier les niveaux d'habilitation des analystes ou injecter de faux utilisateurs.
3. **Verification inter-sessions** : la requete `incidents.find({ status: "open" })` retourne l'incident avec le statut mis a jour par `soc_manager` dans la session precedente. Cela confirme que les modifications sont persistees et visibles entre des sessions differentes – comportement normal mais qui illustre l'importance de l'integrite des donnees.

3.9 Checkpoint 3 – Reponses

Checkpoint – Reponses

Q1 – Difference entre role builtin et role personnalise. Cas concret ?
Un **role builtin** est predefini par MongoDB et couvre des cas d'usage generiques. Par exemple, `read` accorde la lecture sur toutes les collections d'une base, et `readWrite` accorde lecture et ecriture sur toutes les collections sans exception. Ces roles sont pratiques pour un deploiement rapide mais violent le principe du moindre privilege des qu'une base contient des collections de sensibilite differente.

Un **role personnalise** est defini par l'administrateur via `db.createRole()` avec une specification precise de chaque privilege : quelle base, quelle collection, quelles actions. Il peut combiner plusieurs ressources avec des actions differentes et ne couvre que ce qui est explicitement liste.
Cas concret observe : le role builtin `read` sur `soc_lite` donne acces a `users` (donnees personnelles et niveaux d'habilitation), alors que l'analyste ne devrait consulter que les logs et les incidents. Un role personnalise `socAnalyst` avec des privileges sur `security_logs` et `incidents` uniquement aurait empeche cet acces non voulu.

Q2 – Le compte soc_analyst_ro est compromis. Quelles donnees sont exposees ? Quelles donnees restent protegees ?
Donnees exposees : avec le role `read` builtin sur `soc_lite`, l'attaquant peut lire toutes les collections de la base : `security_logs` (tous les evenements de securite historiques), `incidents` (tous les incidents en cours et clos, avec les assignations), et `users` (emails et niveaux d'habilitation des analystes).
Donnees protegees : l'attaquant ne peut pas modifier ni supprimer de donnees (integrite preservee), ne peut pas acceder a d'autres bases (`admin`, `config`, `local`), et ne peut pas administrer le serveur. La portee de la compromission est limitee a la confidentialite des donnees de `soc_lite`.

Q3 – Pourquoi ne pas utiliser adminSoc (root) pour l'application en production ?
Le compte `root` dispose de tous les privileges sur toutes les bases et le cluster. En cas de fuite des identifiants (code source commite par erreur sur un depot public, fichier de configuration mal protege, log d'application qui affiche l'URI de connexion), l'attaquant obtient un controle total : lecture et modification de toutes les bases, creation de nouveaux comptes administrateurs, modification de la configuration serveur, execution de commandes systeme via les fonctions JavaScript de MongoDB. En utilisant `soc_app` avec uniquement `readWrite` sur `soc_lite`, la portee d'une compromission est reduite aux seules operations de collecte de logs. C'est le principe du confinement par le moindre privilege.

4. Synthese – Matrice des droits et recommandations

4.1 Matrice des droits RBAC observes

Operation	soc_analyst_ro	soc_manager	soc_app	adminSoc
Lecture security_logs		×		
Ecriture security_logs	×	×		
Lecture incidents				
Ecriture incidents	×			
Lecture users	!	×	!	
Ecriture users	×	×	!	
Administration cluster	×	×	×	

TABLE 4.1 – Matrice RBAC observee experimentalement. ! = acces non intentionnel du aux roles builtin.

4.2 Recommandations de securite

1. **Remplacer les roles builtin par des roles personnalises** : les roles `read` et `readWrite` sur une base entiere violent le moindre privilege. En production, creer des roles specifiques par collection pour `soc_analyst_ro` et `soc_app`, a l'image de ce qui a ete fait pour `soc_manager`.
2. **Activer TLS/SSL** : l'authentification SCRAM protege les credentials, mais le trafic (donnees) circule en clair par defaut. En production, activer TLS pour chiffrer toutes les communications entre l'application et MongoDB.
3. **Audit logging** : activer le systeme de logs d'audit MongoDB (`-auditLog`) pour

enregistrer toutes les operations d'authentification, de creation d'utilisateurs, et les acces aux collections sensibles. Ces logs alimenteraient eux-memes le SIEM SOC.

4. **Rotation des mots de passe** : les mots de passe utilises dans ce TP (ex : `Passw0rd_S3cure!`) ne doivent jamais etre hardcodes en production. Utiliser un gestionnaire de secrets (HashiCorp Vault, AWS Secrets Manager) et mettre en place une rotation reguliere.
5. **Separation de la base users** : deplacer la collection `users` dans une base separee (`soc_directory`), isolant ainsi completement les donnees personnelles des donnees operationnelles SOC. Les applications qui n'ont pas besoin d'acceder aux utilisateurs ne peuvent alors pas le faire, meme avec des roles builtin.

4.3 Auto-check final

Observation – Validation des trois activites

La connexion anonyme a bien ete refusee sur l'instance secure avec les erreurs `requires authentication`.

`db.getUsers()` affiche `adminSoc` avec le role `root` dans la base `admin`, mecanismes SCRAM-SHA-1 et SCRAM-SHA-256.

L'authentification avec `/soc_lite` echoue, celle avec `/admin` reussit – confirmation de l'importance de l'authenticationDatabase.

Le role personnalise `incidentManager` cree avec succes (`ok: 1`).

`soc_analyst_ro` peut lire logs et incidents mais ne peut pas inserer ni supprimer.

`soc_manager` peut modifier les incidents mais n'a aucun acces a `security_logs` ni a `users`.

`soc_app` peut inserer dans les logs et lire les incidents.

! Limitation identifiee : `soc_analyst_ro` et `soc_app` ont acces non intentionnel a la collection `users` via les roles builtin. Correction a implémenter via des roles personnalises.